Gemini said

In a high-stakes data environment, managing database transactions is about ensuring **Atomicity** and **Consistency**. In Spring, the @Transactional annotation handles this through **AOP (Aspect-Oriented Programming)** by wrapping your method in a proxy that manages the begin, commit, and rollback logic.

---

1. How @Transactional **Works (The Proxy)**

When you annotate a method, Spring creates a **Proxy** around your bean.

- When the method is called, the proxy starts a transaction.
- If the method finishes successfully, the proxy calls commit.
- If an exception occurs, the proxy calls rollback.

Crucial Interview Note: @Transactional only works when the call comes from **outside** the bean. If methodA() calls methodB() within the same class, the proxy is bypassed, and the transaction logic in methodB() will be ignored. This is known as **Self-Invocation**.

---

2. Propagation Basics

Propagation defines the business logic for how a transaction should behave if one is already running when the method is called.

REQUIRED (Default)

- **Behavior:** If a transaction exists, join it. If not, create a new one.
- **Use Case:** The most common. It ensures that all steps in a process succeed or fail as a single unit.

REQUIRES_NEW

- **Behavior:** Always suspends the current transaction and starts a **new, independent** one.
- **Use Case: Audit Logging**. You want to save a record that a user *attempted* an action, even if the main action fails and rolls back.

MANDATORY

- **Behavior:** Must be called within an existing transaction. If none exists, it throws an exception.
- **Use Case:** Internal helper methods that should never be executed without a parent process.

---

3. Rollback Behavior

This is where many developers get caught during technical rounds. Spring's default rollback behavior is very specific.

The Default Rule

By default, @Transactional only rolls back for **Unchecked Exceptions** (RuntimeException and Error). It **does not** roll back for **Checked Exceptions** (like IOException or SQLException).

Customizing Rollback

If you want to ensure a rollback occurs regardless of the exception type, you must specify it: @Transactional(rollbackFor = Exception.class)

The "Catch" Trap

If you catch an exception inside your @Transactional method using a try-catch block and don't re-throw it, Spring will assume everything is fine and **commit** the transaction. To trigger a rollback while catching, you must manually mark it: TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();

---

4. Scenario: The "Batch Migration"

The Situation: You are moving logs from an on-premise system to a cloud database. You have a main method migrateAll() and a child method migrateSingleLog().

- **Requirement:** If one log fails, it shouldn't stop the whole migration.
- **The Technical Setup:**
    1. migrateAll() is marked @Transactional(propagation = Propagation.REQUIRED).
    2. migrateSingleLog() is marked @Transactional(propagation = Propagation.REQUIRES_NEW).

3. **The Result:** When a single log fails, its private transaction rolls back, but migrateAll() catches that specific error and continues to the next log.

Summary for Your Interview

Feature	Key Concept
Proxy Pattern	Self-invocation (internal calls) breaks @Transactional.
REQUIRED	Joins current transaction; one failure rolls back everything.
REQUIRES_NEW	Independent transaction; failure here doesn't affect the caller.
Rollback Rule	Only rolls back for RuntimeException by default.

Final Technical Prep

We have covered the entire "Java Backend" stack: **OOP, Collections, JVM, Threads, Spring Core, and Spring Data.** Since your interview is for a role involving **system stability and migration**, you are now equipped to discuss not just how to write code, but how to ensure that code is resilient and enterprise-ready.